

ANFORDERUNGSANALYSE FÜR ENTWICKLER, TEIL 6

Inkremente innerhalb von Interaktionen finden

Die Anforderungsanalyse muss grob beginnen. Wie aber gelangen Sie zu den kurzen Iterationen, die für schnelle Rückmeldungen im Rahmen agilen Vorgehens unerlässlich sind?

Dieser Abschluss der Serie zur Anforderungsanalyse für Entwickler ([1] bis [5]) zeigt Ihnen, wie Sie die Inkremente, in die Sie Anforderungen beim Slicing schneiden, so fein machen können, dass Sie innerhalb weniger Stunden Feedback zu Ihrer Umsetzung erhalten.

Bei der Zerlegung der Anforderungen mit Slicing geht es zunächst um das „Ob oder ob nicht?“. Soll es ein Inkrement überhaupt geben oder nicht? Und: Ist ein Inkrement schon erkannt oder nicht?

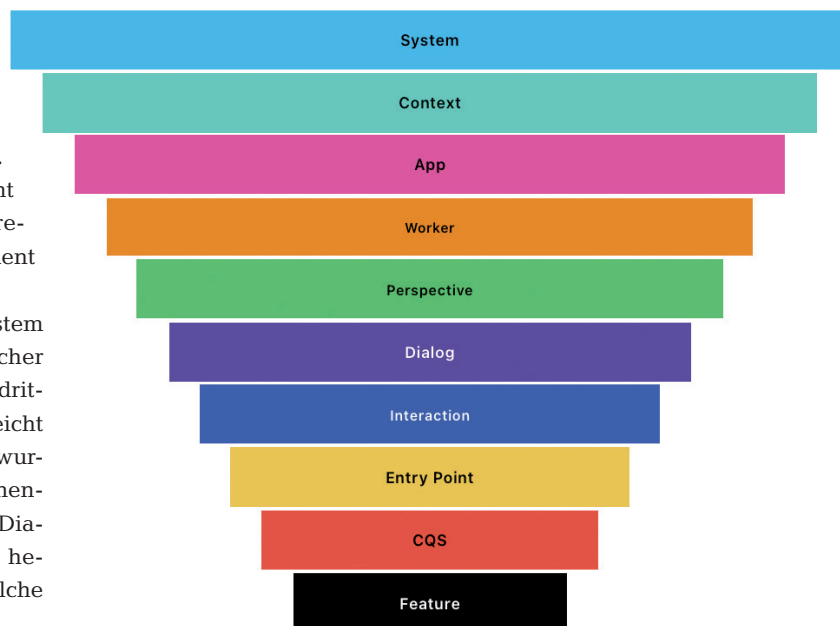
Wo zum Beispiel zwei Kontexte in einem Softwaresystem identifiziert wurden, kann die Frage gestellt werden, welcher davon zuerst begonnen werden soll. Ob es noch einen dritten Kontext gibt, ist vielleicht noch unklar. Wo dann vielleicht zehn Dialoge innerhalb einer Perspektive identifiziert wurden, stellt sich die Frage, welche davon in welcher Reihenfolge umgesetzt werden sollen. Sobald über diese zehn Dialoge hinweg zum Beispiel insgesamt 75 Interaktionen herausgearbeitet worden sind, kann überlegt werden, welche davon überhaupt und wann umgesetzt werden.

Und „ganz unten“ geht es darum, welche Funktionen auf der CQS-Ebene implementiert werden sollen. Die sind natürlich niemals für ein komplettes Softwaresystem mit seinen Applikationen, Workern und Dialogen durch eine initiale Analyse erkennbar. Sie ergeben sich vielmehr über die Zeit. Immer wieder gehen Sie zurück in die Anforderungsanalyse und fragen: „Was noch? Was fehlt?“ Als Antwort schälen sich im Dialog mit dem Kunden weitere Entry Points heraus.

Existenzielle Inkremente

In Bild 1 sehen Sie, wie eine Erkundung des Anforderungsraums – oder konkreter: des Anforderungsbaums – sowohl in der Breite als auch in der Tiefe verläuft. Sie beginnt mit einer Breadth-first-Phase, die einen Überblick verschafft: Welche größeren Brocken müssen eigentlich bewältigt werden? Wo liegen architektonische Herausforderungen? Dazu gehören die Ebenen System, Context, App und Worker.

Ab einem bestimmten Punkt jedoch ist es interessant, sich zu konzentrieren und einen Ast in die Tiefe zu verfolgen bis zur konkreten Funktion mit Signatur und Testfällen. Innerhalb von Workern geht es dabei depth-first vorwärts über Perspektiven, Dialoge, Interaktionen, Entry Points und CQS-Funktionen.



Depth-first ist so aufwendig, dass diese Phase nicht vollständig in der Breite durchlaufen werden kann; nur für eine gewisse Zeit ist eine Beschränkung auf breadth-first möglich. In der Breite bleiben daher über kurz oder lang Lücken. Inkremente werden nicht weiter verfolgt; darunter hängende Inkremente werden nicht einmal entdeckt.

Das meine ich mit „Ob oder ob nicht?“. Es kann eben nicht alles in der Tiefe realisiert werden; auf einmal sowieso nicht. Es kann jedoch noch nicht einmal alles in der Tiefe erkannt werden.

Die Natur der Softwareentwicklung ist also nicht nur inkrementell: ein Inkrement nach dem anderen realisieren. Sie ist auch notwendig iterativ: Die Analyse muss mehrfach durchlaufen werden, um ein Softwaresystem immer näher an einen Zielzustand heranzubringen. (Natürlich ist dieser Zustand auch noch ein bewegliches Ziel, was die Notwendigkeit zur Iteration verdoppelt.)

Die bisherigen Inkremente könnte ich existenziell nennen. Sie existieren entweder oder eben nicht. Der Entry Point $ep(x:string):number$ wird entweder realisiert oder nicht. Dafür muss es die Interaktion i im Dialog d in der Perspektive p

im Worker w in der App a im Kontext k im Softwaresystem geben. Falls irgendetwas davon nicht aufgesetzt ist, braucht es keine Funktion $ep()$.

Oder genauer: Mindestens muss es die Hierarchieebenen konzeptionell darüber geben, weil ihre Herausarbeitung zur Erkenntnis geführt hat, dass $ep()$ nötig ist. Sie müssen nicht unbedingt implementiert werden, um $ep()$ zu realisieren und gegen Akzeptanztests laufen zu lassen. Dasselbe gilt, wenn $ep()$ eventuell in die CQS-Funktionen $update(x:string):void$ und $load():number$ zerlegt wurde.

Bis hierhin dreht sich Slicing um die Zerlegung in existenzielle Inkremente. Und die ganze Zerlegung hat das Ziel, die Entry-Point-Funktionen als Inkremente zu identifizieren,

Zerlegungshierarchie, das heißt eines ihrer Blätter, kann noch höchst umfangreich sein.

Es wäre deshalb wünschenswert, weiter zerlegen zu können. Sie wollen einem PO nicht sagen, „Gut, dass wir im Entry Point noch das Command $update()$ und die Query $load()$ gefunden haben. Du kannst in sechs Wochen mit der Umsetzung von $update()$ rechnen.“ Das wäre gänzlich unagil. Inkremente dürfen nicht Wochen für die Realisierung brauchen. Eine Rückmeldung vom PO muss in viel, viel kürzeren Zeitabständen eingeholt werden können.

Leider sehe ich jedoch keine simple weitere formale Zerlegbarkeit von Inkrementen wie vom System bis zur CQS-Funktion. Funktionen sind etwas anderes als Dialoge; die

breadth-first

depth-first

Anforderungsanalyse mit Slicing in zwei grundsätzlichen Phasen: zuerst in der Breite, dann in der Tiefe (Bild 1)

weil diese für Sie konkrete Einstiegspunkte in die Umsetzung mit Entwurf und Codierung sind. Erreicht ist dieser Level erst, wenn Sie sich über die Existenz der Funktionen im Klaren sind, Sie deren Signaturen klar herausgearbeitet haben und der PO/Kunde sich klar über Testfälle ist. Vorher sollten Sie nicht mit der Arbeit an Produktionscode beginnen. Das Risiko für Änderungen, die seine Sauberkeit belasten, ist einfach zu groß. Unentschiedenes Hin und Her auf Produktionscode ist Gift für die Langlebigkeit einer Codebasis.

Qualitative Inkremente

Auch wenn die Zahl solcher existenziellen Inkremente in einem Softwaresystem sehr, sehr groß ist, bedeutet das nicht, dass deshalb der Scope dieser Inkremente auf der untersten Ebene klein ist. Selbst wenn Sie schließlich innerhalb eines Entry Points einer Interaktion innerhalb eines Dialogs innerhalb einer Perspektive innerhalb eines Workers innerhalb einer App in einem Kontext eines Softwaresystems auf ein Kommando gestoßen sind, stehen Sie nicht wegen all der harten Zerlegungsarbeit vor sehr feinen Anforderungen, die im Nu umgesetzt sind. Nein, auch das Inkrement am Ende der

wiederum sind etwas anderes als Worker (Betriebssystemprozesse); diese unterscheiden sich schließlich von Kontexten als durch Kontrakte definierte Sprachgrenzen.

Unterhalb von (CQS-)Funktionen jedoch... Was soll da noch kommen? Es können nur weitere Funktionen sein. Entry Points sind Wurzeln von Funktionsbäumen, die zur Laufzeit das Verhalten herstellen, das Benutzer von einer Interaktion erwarten.

Für mich ist jedoch weniger klar, welche Funktionen es in diesem Funktionsbaum geben sollte. Eine weitere Zerlegungshierarchie in Form einer Checkliste wie die der Slicing-Ebenen bis hierher sehe ich nicht. Jetzt wird es schwieriger. Das Ziel ist nun nicht mehr so deutlich vor Augen wie mit Entry Points und ihren Verfeinerungen, den Commands und Queries.

Deshalb würde ich nicht mehr von existenziellen Inkrementen sprechen, sondern von qualitativen Inkrementen.

Durch eine weitere Verfeinerung der Anforderungen innerhalb und unterhalb eines Entry Points geht es um dessen qualitative Ausprägung. Ein Beispiel mag verdeutlichen, was ich meine. ►

Nehmen Sie eine Aufgabenverwaltung als zu realisierendes Softwaresystem an. Das ist sehr übersichtlich und lässt sich naheliegendermaßen wie folgt zerlegen:

- Das ganze System steht für die Aufgabenverwaltung.
 - Darin gibt es eine Desktop-App.
 - Die App hat einen Dialog für die Erfassung der Details zu einer Aufgabe.
 - Vor allem hat sie aber einen Dialog für die Übersicht über alle Aufgaben mit folgenden Interaktionen:
 - Hinzufügen einer neuen Aufgabe
 - Bearbeiten einer Aufgabe
 - Abhaken einer Aufgabe
 - Löschen einer Aufgabe
 - Wahl zwischen Anzeige unerledigter und erledigter Aufgaben
 - Öffnen des Dialogs mit Anzeige aller Aufgaben
 - Der Entry Point sieht dafür so aus:
`loadAll():Task[]`. Das ist eine Query;
 Verfeinerung auf CQS-Ebene ist nicht nötig.

Für den letzten, explizit genannten Entry Point kann ein Test formuliert werden; Sie können mit der Umsetzung beginnen. Dasselbe ist aber auch leicht für die Entry Points der anderen Interaktionen möglich, die ich hier der Einfachheit halber ausgelassen habe. So weit dreht sich alles um existenzielle Inkremente.

Wann könnten Sie nun dem PO eine Umsetzung für eine Rückmeldung vorlegen? Wovon hängt der Umsetzungsaufwand ab? Zunächst hängt Ihr Aufwand davon ab, wie viele der existenziellen Inkremente Sie bis zu einer Rückmeldung umsetzen wollen/sollen. Ist der PO nur mit allen Entry Points zufrieden? Oder kann er sich auf eine Untermenge einlassen, gar nur einen Entry Point? Je weniger es sind, desto schneller sind Sie, oder?

Die Menge der existenziellen Inkremente ist die eine Dimension, in der Sie einen Regler bewegen können, um die Zeit bis zu einer Rückmeldung zu verändern.

Eine zweite Dimension ist die Qualität, in der die existenziellen Inkremente umgesetzt werden. Damit meine ich zunächst die für den PO fühlbare Qualität. Wie ließe sich die im Beispiel unterschiedlich gestalten? Zum Beispiel könnte eine erste Umsetzung ohne Persistenz auskommen. Alle geforderten Entry Points wären vorhanden, die App könnte bedient werden – allerdings würden die Daten lediglich in-memory gehalten. Das könnte Ihren Implementationsaufwand drastisch senken.

Die Qualität wäre geringer als die gewünschte, doch es wäre ein Schritt in die richtige Richtung. Viele Aspekte des Softwaresystems

könnten vom PO überprüft werden, ohne dass der Aufwand für Persistenz getrieben werden müsste.

Woran würde der PO merken, dass diese Qualität fehlt? Wenn er die App schließt und wieder öffnet, sind vorher erfasste Aufgaben verschwunden.

Wie könnten Sie die Qualität der App ein wenig steigern und dem PO schon einen Eindruck von ihrer Arbeitsweise mit Persistenz verschaffen? Sie könnten die interne Liste mit den Aufgaben einfach pauschal beim Schließen der App zum Beispiel im JSON-Format in einer Textdatei speichern und beim Start von dort wieder laden.

Woran würde der PO merken, dass immer noch Qualität fehlt? Er könnte nicht in zwei Instanzen der Anwendung auf demselben Rechner oder auf verschiedenen Rechnern gleichzeitig an den Aufgaben arbeiten. Die letzte geschlossene Instanz würde mit ihren Daten „gewinnen“.

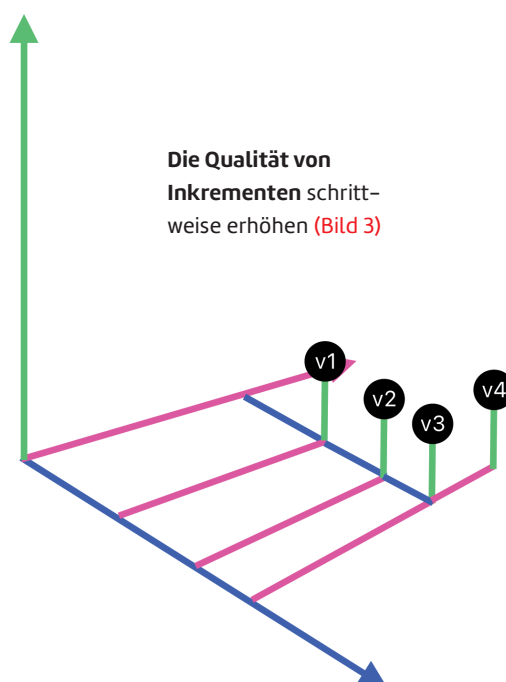
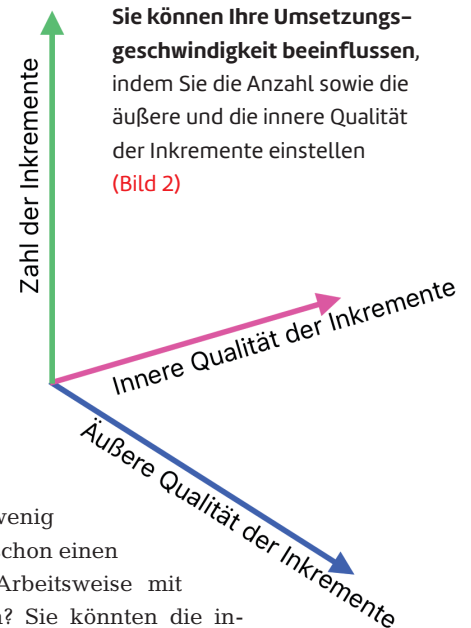
Das meine ich mit qualitativen Inkrementen. Sie beschreiben verschiedene „Ausbaustufen“ des Verhaltens, das durch einen Entry Point repräsentiert wird.

Die Dimensionen der Qualität

Genau genommen gibt es sogar zwei Qualitätsdimensionen (Bild 2): Die eine steht für fühlbare Qualität wie im Beispiel; es geht um Softwareverhalten. Die andere steht für nicht fühlbare Qualität; PO oder Kunde oder Anwender können

keine Aussage im Moment der Benutzung darüber machen. Nur über eine längere Projektlaufzeit würden sie bemerken können, dass es an einer gewissen Qualität mangelt, obwohl äußerlich alles läuft, wie es soll. Mit dieser anderen Qualität meine ich die innere, das heißt die, um die es auch beim Clean Code Development geht. Sie ist gekennzeichnet durch Testbarkeit und Wandelbarkeit.

Auch die innere Qualität können Sie bei Ihrer Umsetzung einstellen; Sie können sich für mehr oder für weniger entscheiden. Ausdruck dafür sind zum Beispiel die Testabdeckung oder der Grad der Modularisierung oder ob Sie Prinzipien sauberer Codierung eingehalten haben.



Sie können hohe äußere Qualität (zum Beispiel alle Entry Points laufen mehrbenutzerfähig persistent) und geringe innere Qualität (zum Beispiel sie weisen eine geringe Testabdeckung auf) haben; Sie können aber auch niedrige äußere Qualität (zum Beispiel alle Entry Points laufen nur mit In-Memory-Datenhaltung) und hohe innere Qualität (zum Beispiel Sie haben hohe Testabdeckung und saubere Modularisierung) kombinieren.

Ganz grob empfehle ich jedoch das Vorgehen wie in **Bild 3** skizziert:

- Wählen Sie nur eine kleine Zahl existenzieller Inkremente,
- implementieren Sie diese zuerst mit einer geringeren äußeren Qualität
- und auch nur mit suboptimaler innerer Qualität (v1).
- Dann addieren Sie äußere Qualität in mehreren Schritten, bis alle (bekannten) Wünsche erfüllt sind (v2, v3).
- Zum Schluss ziehen Sie die innere Qualität „gerade“. Sie refaktorisieren und runden Ihre Testsuite ab (v4).

Bei gegebener Zahl der existenziellen Inkremente – idealerweise konzentrieren Sie sich stets nur auf einen Entry Point zur gleichen Zeit –, sieht die Kurve der Qualitätsentwicklung sogar eher wie in **Bild 4** aus: Sie starten mit suboptimaler innerer Qualität, diese sinkt im Verlauf der Iterationen, durch die Sie für das existenzielle Inkrement laufen, und erst am Ende erhöhen Sie sie auf den eigentlich notwendigen Level.

Realistisches Clean Code Development erkennt an, dass äußere und innere Qualität nicht gleichzeitig gesteigert werden können. Ja, innere Qualität kann nicht einmal stets gehalten werden, während an der äußeren Qualität gearbeitet wird. Deshalb braucht es immer wieder „Sprünge“ nach oben für die innere Qualität, während derer sich die äußere nicht verändert; das sind Refaktorisierungen.

Bitte beachten Sie, dass auch die Steigerung der inneren Qualität ein Inkrement darstellt. Selbst wenn PO, Kunde und Anwender davon nichts (unmittelbar) bemerken, wird der Wert der Software dadurch gesteigert. Und um nichts anderes geht es bei der inkrementellen Entwicklung von Software: die Steigerung ihres Wertes in kleinen Schritten.

Wert versus Erkenntnis

Wenn ich von Wert schreibe, denken Sie sicherlich an Wert für den Kunden. Der drückt sich letztlich in Mehrgewinn aus: Wenn Software ein bestimmtes Verhalten zeigt, das für Benutzer nützlich ist, mehrts das den Gewinn des Unternehmens auf die eine oder andere Weise mehr oder weniger unmittelbar. Nützliches Verhalten senkt die Kosten und/oder steigert den Umsatz. Aus anderen Gründen sind Unternehmen nicht gewillt, Geld für Software auszugeben.

Dass sie für diese Nützlichkeit Geld ausgeben müssen, ist selbstverständlich. Diese Kosten werden mit dem Mehrgewinn verrechnet. Wenn Aussicht besteht, dass etwas übrig bleibt, könnte sich das Softwareprojekt lohnen, wenn nicht ein besserer ROI eines anderen Projekts oder geringere Opportunitätskosten woanders dem entgegenstehen.

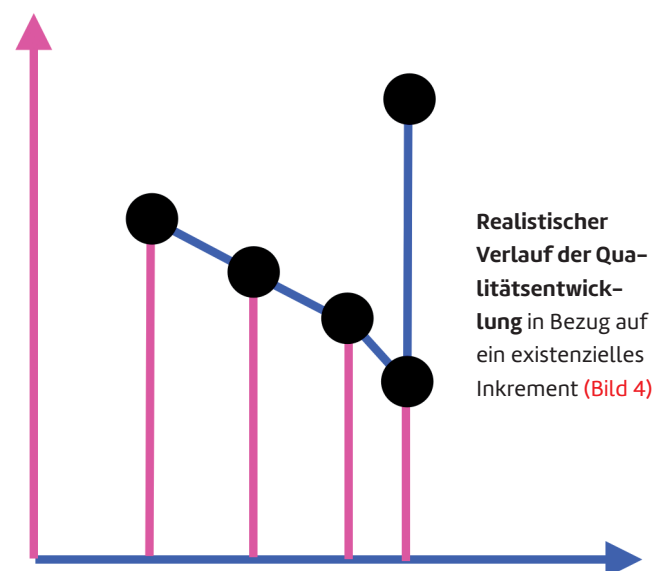
Aber wir wollen uns nicht in Details der Wirtschaftlichkeitsbetrachtung verlieren. Es geht mir nur darum, klarzumachen,

dass Wert für Geld steht. Und damit stehen Inkremente für Geld. Denn Inkremente sollen Wert liefern. Ein PO ist aufgerufen, die Wertsteigerung durch geschicktes Schneiden und Priorisieren von Inkrementen zu optimieren.

Bei den sprichwörtlichen Wasserfallprojekten entsteht Wert erst ganz am Schluss mit einem großen Sprung (oder vielleicht auch bei Erreichen gewisser Meilensteine). Bis dahin vergeht viel Zeit, in der das Softwareprojekt nur Geld kostet; währenddessen ist unsicher, ob und wie viel Wert wirklich demaleinst entstehen wird.

Dem stellt die Agilität eine Vorgehensweise entgegen, die viel häufiger Wert liefert. Wird nach Scrum gearbeitet, könnte das zum Beispiel alle vier oder alle zwei Wochen sein. Wer nach Kanban arbeitet, liefert mit jedem Inkrement, das auf „Done“ gestellt wird, Wert; das kann jeden Tag passieren.

Hier ein Beispiel: Ein PO hat vier Inkremente identifiziert, die ab Release jeden Monat einen bestimmten Mehrgewinn generieren. Jedes Inkrement braucht natürlich auch eine gewisse Umsetzungszeit. In welcher Reihenfolge sollten die Inkremente angegangen werden, um eine möglichst große Wertsteigerung zu erzielen? **Bild 5** stellt zwei mögliche Priorisierungen gegenüber. Eine erzielt nach zwölf Monaten 14 400 Euro Mehrgewinn, die andere 15 500 Euro.



Während der Kunde für den Wert eines ganzen Softwaresystems sicher eine Vorstellung hat, weil er es ja bewilligen muss und also die Kosten gegen Alternativen abwägt, sieht es für feinere Inkremente schon anders aus. Kontext oder Apps, also „dicke Scheiben“ des Slicings, mögen auch noch einen bezifferbaren Wert haben, Dialoge oder Interaktionen hingegen kaum.

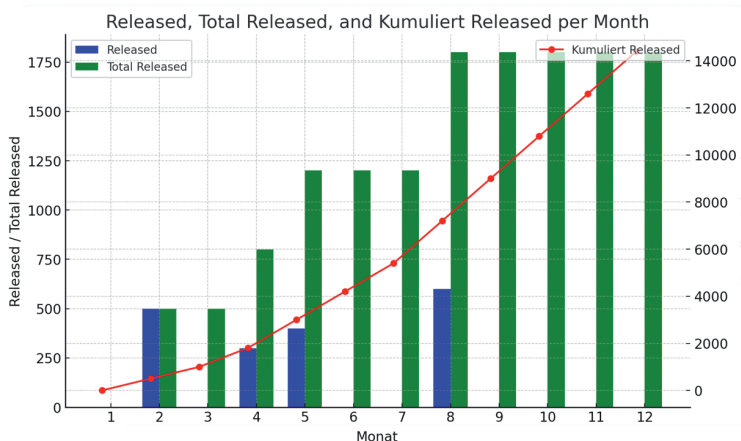
Dass ein PO wüsste, welchen Mehrgewinn die Umsetzung dieses oder jenes Entry Points generieren würde, ist kaum anzunehmen. Geschweige denn, dass ein PO das für einzelne Qualitätsaspekte innerhalb eines solchen Inkrements sagen könnte. Dennoch wird ein PO irgendeinen Eindruck davon haben, in welchem Verhältnis die Werte mehrerer ►

Inkmente stehen. Wenn ein genauer Geldbetrag nicht bezifferbar ist, ist das nicht so schlimm. Auch ohne exakte Zahl ist aber klar, dass Wert sich um Geld dreht.

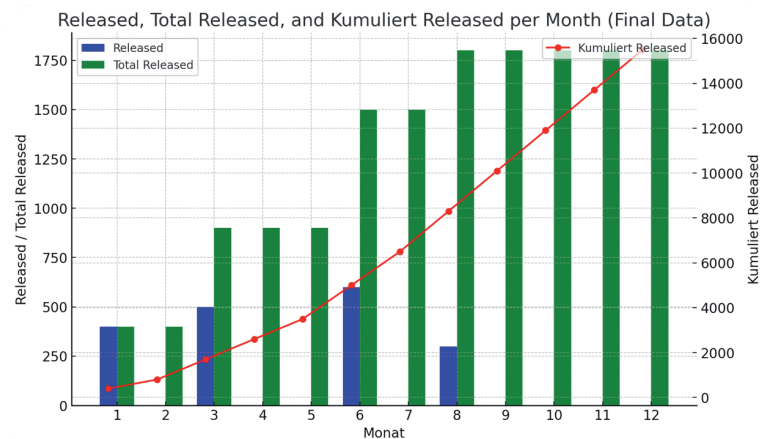
Wie passt nun dazu, dass ich oben Qualitätsinkremente vorgeschlagen habe, die offensichtlich keinen Wert für den Kunden haben. Wenn Sie eine Aufgabenverwaltung zunächst mit geringer äußerer Qualität realisieren, indem Sie

das schneller geradegezogen werden, als wenn er erst später höherqualitative Inkremente in die Hand bekommen hätte.

Damit möglichst schnell Wert hergestellt wird, ist möglichst schnell festzustellen, was überhaupt wertvoll ist. Das weiß ein PO nicht immer a priori. In solchem Fall zu versuchen, ein Wertinkrement herzustellen, das notwendig zu kurz springen muss, ist Verschwendung. Besser ist es, bei der Qualität ab-



Die Priorisierung macht den Unterschied bei der Generierung von Wert. Blaue Balken: Am Anfang eines Monats geliefert Wert; grüne Balken: Wert, der pro Monat durch alle bisherigen Lieferungen entsteht; rote Kurve: kumulierter Wert. Unterschiedliche Priorisierungen führen zu unterschiedlichem Anstieg der Wert-Kurve (Bild 5)



Daten nur in-memory halten, können Anwender damit nichts anfangen. Sie haben ein Inkrement geliefert und keinen Wert hergestellt?

Ich bin der Meinung, das passt. Inkremente sollen möglichst häufig möglichst viel Wert mit wenig Aufwand liefern – doch mit diesem Anspruch kann es lange dauern, bis Sie etwas zur Rückmeldung vorlegen. Deshalb unterscheide ich zwischen Wert-Inkrementen und Erkenntnis-Inkrementen.

- Wert-Inkremente können ausgeliefert werden und erzeugen Mehrgegninn für den Kunden.
- Erkenntnis-Inkremente sind nicht auslieferungsfähig, sondern nützen vor allem dem internen Erkenntnisgegninn. Sie beziehungsweise der PO haben davon einen Vorteil.

Ein Erkenntnis-Inkrement dient zur Orientierung: „Sind wir auf dem richtigen Weg zum Wert?“

Bei reduzierter äußerer Qualität bekommt ein PO allerdings immer noch etwas in die Hand, woran er feststellen kann, ob seine Idee von Anforderungen beziehungsweise Ihr Verständnis und Ihre Umsetzung wirklich den Nutzen für den Kunden erhöhen würden.

Beispiel Aufgabenverwaltung mit In-Memory-Datenhaltung: Darin steckt kein Wert, dennoch erlaubt sie dem PO einen Umgang mit der App, der ihm Erkenntnis darüber verschaffen kann, ob die Darstellung so ist, wie er sie sich wünscht, und die Interaktionen so erreichbar und granular sind, wie sie für bequeme Nutzung sein sollen. Er bekommt ein End-to-End-Benutzererlebnis – nur ohne Persistenz. Falls ihm dabei etwas auffällt, das verändert werden sollte, kann

zuspecken und vor allem das zu liefern, was viel Licht ins Anforderungsdunkel bringt, spricht Erkenntnis. Ist die Erkenntnis gewonnen, kann mehr Qualität in einer nächsten Iteration geliefert werden – bis schließlich sogar Wert entsteht.

„Nur“ ein Erkenntnis-Inkrement zu liefern und noch kein Wert-Inkrement, ist mithin ökonomisch. Ein Mangel wird möglichst schnell ausgeglichen, teure Umwege werden vermieden.

Und woher weiß der PO, ob er noch nicht genau weiß, was für den Kunden welchen Wert darstellt? Er weiß es oft eben nicht, höchstens ahnt er es. Auch Sie wissen es nicht. Deshalb lautet meine Empfehlung, Inkremente so klein wie möglich zu schneiden und Erkenntnis-Inkremente strategisch an den Anfang zu stellen. Suchen Sie offensiv nach Wegen, zuerst Erkenntnis zu gewinnen und nicht sofort dem Wert hinterherzuhecheln. Der Wert wird sich einstellen, wenn die Erkenntnis solide gewonnen ist.

Maximal 16 Stunden bis zur Erkenntnis

Der Zweck hinter Slicing ist nicht nur, auf kristallklar definierte Ausgangspunkte für Entwurf und Codierung inklusive automatisierte Tests zu kommen: Entry Points. Er besteht auch darin, Anforderungen so fein zu schneiden, dass zügig Feedback gewonnen werden kann.

Was ist zügig? Meine Definition ist sehr einfach: maximal 16 Stunden Arbeitszeit.

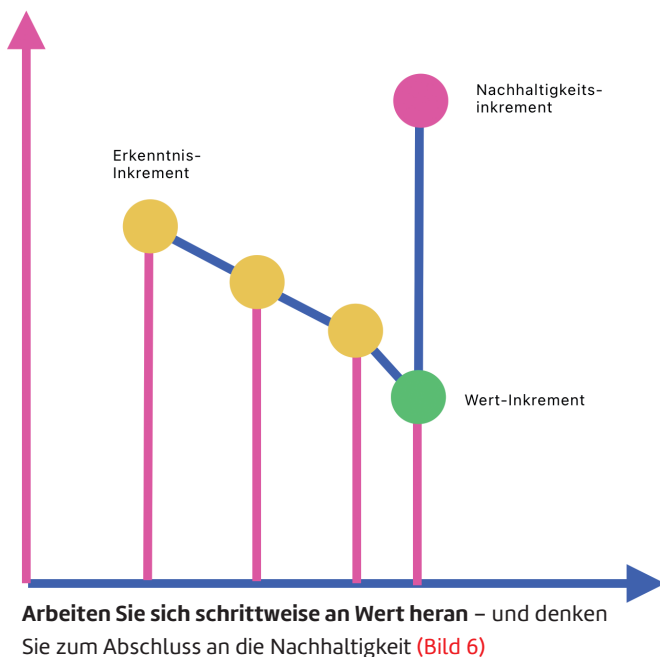
Vom Beginn der Umsetzung eines Inkrements bis zu einer Lieferung zwecks Rückmeldung vom PO sollten nicht mehr als 16 Arbeitsstunden vergehen. Wenn Sie heute um 9:00 Uhr

mit der Umsetzung beginnen, sollten Sie morgen um 17:00 Uhr damit fertig sein. Natürlich darf es auch schneller gehen, nur länger sollte es nicht dauern.

Kürzer, schneller ist sehr schwierig. Schon 16 Arbeitsstunden werden Ihnen für viele Inkremente unrealistisch erscheinen. Zu recht, wenn Sie nur an Wert-Inkremente denken. Aber um die geht es ja nicht mehr ausschließlich. Erkenntnisgewinn sollte in 16 Arbeitsstunden möglich sein.

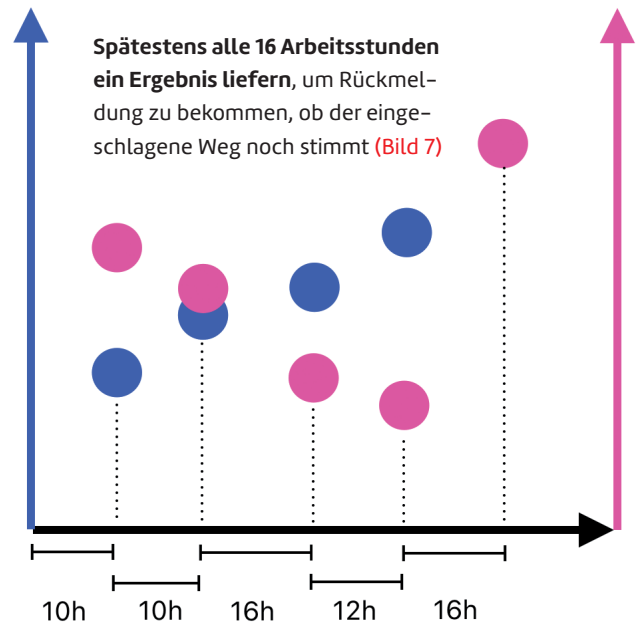
Und wenn nicht, sagt das auch etwas über den Stand der Kenntnis bei PO und/oder bei Ihnen aus: Sie bewegen sich in großer Unsicherheit. Umwege, also Verschwendung drohen; Frustration, Unzuverlässigkeit, Qualitätsmängel, Konflikte liegen vor Ihnen – es sei denn, Sie packen diese Unsicherheit bei den Hörnern. Prototypen und Spikes mögen dann bessere Mittel sein als Inkremente, die Produktionscode verändern.

Länger als 16 Arbeitsstunden ist natürlich sehr einfach. Nur droht dann erstens „Stochern im Nebel“ angesichts fehlender Rückmeldung, zweitens steigt das Risiko von Multitasking. Man wird Sie kaum zehn Tage lang ungestört an einem Wert-Inkrement werkeln lassen – dessen Wert am Ende trotz aller Mühe ungewiss ist. Vielmehr wird man Ihnen immer wieder andere Aufgaben/Inkremente dazwischenschieben; ja, auch und gerade bei agilem Vorgehen. Auf diese Weise dehnt sich das 10-Tage-Inkrement schnell auf 12, 15, 20 Tage aus. Die Rückmeldungen entschwinden in unbekannte Zukunft. Das macht dann auch irgendwann keinen Spaß mehr.



Nein, Sie wollen so schnell wie möglich Klarheit. Eine Rückmeldung – selbst wenn Sie negativ sein sollte – ist auch wichtig für Ihren „sense of purpose“. Wenn Sie merken, dass da jemand ist, der sich für Ihre Ergebnisse interessiert, sind Sie mit mehr Engagement bei der Sache.

Ich glaube, dass eine Lieferung „spätestens morgen“ hohen Fokus fördert. Zur Not schaffen Sie es nämlich sogar, bis



morgen andere Aufgaben abzulehnen. Über ein oder zwei Tage ist es möglich, sich echt zu konzentrieren, um verlässlich zu liefern. Ist das geschafft, kann erst mal wieder etwas anderes dran sein. Aber zweimal acht Arbeitsstunden können Sie unter Kontrolle haben, wenn Sie es ernst meinen mit der Verlässlichkeit. Mehr scheint mir in einem normalen Projektbetrieb sehr unwahrscheinlich.

Wie gesagt, damit meine ich keine Wert-Inkremente, sondern vor allem Erkenntnis-Inkremente. Verabreden Sie die explizit mit Ihrem PO. Suchen Sie im Rahmen eines Entry Points nach weiteren Inkrementen, die einen schrittweisen Aufbau seines Gesamtverhaltens und damit Gesamtnutzens und Wertes erlauben.

In Bild 6 sehen Sie, wie ich mir die Entwicklung eines Wert-Inkrementes vorstelle: Es ist der Schlussstein eines Weges, der bis dahin mit Erkenntnis-Inkrementen gepflastert ist. Um diesen Wert jedoch langfristig zu erhalten, müssen Sie darauf achten, die innere Qualität anschließend ebenfalls wieder auf einen hohen Wert zu bringen.

Denn gelieferter Wert heute im Sinne von Nützlichkeit interessiert den Kunden eigentlich nur, wenn dadurch zukünftige weitere Nützlichkeitssteigerungen nicht kompromittiert werden. Nützlichkeit braucht Nachhaltigkeit. (Wenn Kunden scheinbar so nicht denken, ist das vor allem ihrem Unverständnis anzurechnen, was die Möglichkeiten des Raubbaus an der Ressource Code angeht. Wüssten Sie darum, würden Sie offensiver hohe innere Qualität fordern.)

Die x-Achse in Bild 6 können Sie übrigens auch zeitlich verstehen: Mit fortschreitender Zeit erhöhen Sie zunächst die äußere Qualität – und addieren am Ende nochmal Aufwand für die innere. In Bild 7 sind die Entwicklungen von innerer und äußerer Qualität je Inkrement parallel notiert.

Single Responsibilities für schnelle Erkenntnis

Was erzeugt Wert? Die Summe der in Code umgesetzten Entscheidungen des Kunden. ►

Der Kunde hat sich entschieden, die Realisierung einer Aufgabenverwaltung zu beauftragen. Das ist ihm zum Beispiel 5000 Euro wert; er erhofft sich dadurch einen Mehrge-
winn von 250 Euro pro Monat. Die Lieferung des Codes für die Aufgabenverwaltung startet die Produktion des Mehrge-
winns.

Auf dieser Abstraktionsebene steht einer Entscheidung ein Inkrement gegenüber.

Diese eine große Entscheidung besteht jedoch aus ande-
ren, dieser Entscheidung müssen weitere folgen. Wenn eine Aufgabenverwaltung entwickelt werden soll, stellen sich zum Beispiel diese Fragen:

- Soll es eine Desktop-, Web- oder Mobile-App sein? Oder sollen womöglich mehrere Endgeräte bedient werden?
- Soll sie mehrbenutzerfähig sein?
- Soll sie für 5, 25, 100, 1000 oder mehr Benutzer ausgelegt sein?
- Sollen andere Softwaresysteme integriert werden (zum Beispiel Google Kalender)?
- Für welches Datenaufkommen pro Benutzer soll das Softwaresystem ausgelegt sein?
- Soll es eine Authentifizierung für Benutzer geben?
- Soll es eine Autorisierung mit unterschiedlichen Rechten für Benutzer geben?
 - Soll es Benutzerrollen geben?
- Welche Funktionalität soll das Softwaresystem haben?
 - Sollen Aufgaben in Listen zusammengefasst werden können?
 - Sollen Aufgaben (listenübergreifend) getaggt werden können?
 - Sollen Tags zentral umbenannt werden können?
 - Soll eine Erinnerung an Aufgaben erfolgen? Wenn ja, wie?
 - Soll der Fortschritt bei der Erledigung von Aufgaben festgehalten werden können?
 - Sollen Aufgaben aufeinander Bezug nehmen können, sodass Abhängigkeitsbeziehungen entstehen (Vorgänger, Nachfolger ...)?
 - und so weiter und so fort.

Sie sehen, es gibt viele, viele, viele Entscheidungen zu tref-
fen. Kunden müssen sich darüber klar werden, was sie wol-
len. POs sollen ihnen helfen, diese Klarheit zu erlangen. Klar-
heit besteht darin, Entscheidungen erstens zu benennen und
zweitens explizit zu entscheiden.

Entscheidungen, die nicht herausgearbeitet sind und nicht
explizit entschieden sind, werden ansonsten von Ihnen als
Entwickler mehr oder weniger bewusst entschieden. Sie ma-
chen Annahmen – deren Umsetzung am Ende dem Kunden
vorgelegt wird. Ob der mit Ihren Annahmen übereinstimmt,
ist fraglich. Änderungsaufwand droht.

Ihre vornehmste Aufgabe während der Anforderungsana-
lyse besteht mithin darin, alle Entscheidungen des Kunden
herauszuarbeiten, explizit zu machen und die vom Kunden
getroffenen Entscheidungen festzuhalten. Ihre Verantwort-
lichkeit ist es anschließend, Code zu schreiben, der diese Ent-
scheidungen umsetzt.

Und das Single Responsibility Principle (SRP) trägt Ihnen
dazu auf, den Code, der die Verantwortung für die Umset-
zung einer Entscheidung trägt, mit einem Bezeichner zu ver-
sehen [6].

Entscheidungen ziehen Verantwortlichkeiten im Code
nach sich. Welche Codeabschnitte für welche Entschei-
dungen verantwortlich sind, ist glasklar zu machen. Nur so kann
Code gezielt angepasst werden, wenn sich die Entschei-
dungen ändern; nur so kann Code verlässlich getestet werden, ob
er überhaupt seiner Verantwortlichkeit gerecht wird.

Das SRP ist also analyseleitend und gleichzeitig auch ent-
wurfsinspirierend. Größere Entscheidungen werden durch
größere Strukturelemente repräsentiert (zum Beispiel Pro-
zess, Bibliothek, Klasse). Kleinere Entscheidungen, Detail-
entscheidungen werden durch Funktionen repräsentiert.
Noch feinere durch einzelne Ausdrücke.

Wenn Sie auf der Suche nach Inkrementen sind, folgen Sie
schon intuitiv einem Entscheidungsbaum in die Tiefe. Jeder
Kontext, jede App, jeder Worker, jeder Dialog und jede Inter-
aktion stehen für eine Entscheidung. Dass zum Beispiel Auf-
gaben gelöscht werden können sollen, ist eine Entscheidung,
für die es eine Interaktion mit einem Entry Point gibt.

Doch da endet der Entscheidungsbaum gewöhnlich nicht.
Er setzt sich in die Tiefe fort. Weitere Entscheidungen machen
in Summe die aus, für die eine Interaktion steht.

Aufgaben sollen gelöscht werden können? Wie steht es
dann zum Beispiel mit den folgenden Fragen:

- Wie soll unbeabsichtigtes Löschen verhindert oder im Ef-
fekt gemildert werden? Soll vor dem endgültigen Löschen
eine Bestätigung eingeholt werden? Oder soll die Aufgabe
nur als gelöscht gekennzeichnet werden, um sie gegebe-
nenfalls über eine Rückgängig-Interaktion wieder herzu-
stellen?
- Soll eine Aufgabe gelöscht werden können, wenn andere
von ihr abhängig sind?
- Wenn eine gelöschte Aufgabe die letzte mit einem Hashtag
ist, soll dann das Tag aus der Liste aller Hashtags ver-
schwinden oder dennoch darinbleiben?
- Soll das Löschen durch einen Soundeffekt begleitet wer-
den?
- Soll das Löschen durch eine Animation begleitet werden?
- Braucht ein Benutzer für das Löschen ein bestimmtes Recht?
 - Kann er eigene Aufgaben immer löschen und nur
fremde mit einem bestimmten Recht?

Diese und weitere Entscheidungen sind herauszuarbeiten
und nach dem SRP klar mit einem Bezeichner identifiziert zu
implementieren. Softwareentwicklung, ja, schon Anforderungs-
analyse ist wahrlich kein leichtes Unterfangen.

Doch diese Sichtweise auf die Analyse hat auch einen Vor-
teil: Jede Entscheidung ist potenziell eine, die auch erst mal
vernachlässigt werden kann. Es kann für den Zweck des Er-
kenntnisgewinns nur die einfachstmögliche Entscheidung
getroffen werden. Sie fällt immer so aus, dass Sie weniger
Umsetzungsaufwand haben. Flüssige agile, also im Kern in-
krementell-iterative Arbeit basiert auf der Kunst des Loslas-
sens. Loslassen von „lieb gewonnenen“ Ideen, was eine Soft-

ware alle können soll. Nicht für immer, aber doch für einen gewissen Zeitraum.

Ein paar Beispiele fürs Loslassen:

- Unbeabsichtigtes Löschen verhindern? Nein.
- Darauf achten, ob es Abhängigkeiten gibt? Nein.
- Hashtags anpassen? Nein.
- Soundeffekt? Nein.
- Animation? Nein.
- Rechte für das Löschen? Nein.

Wer bei der Benutzung die Interaktion „Aufgabe löschen“ triggert, der löscht einfach die Aufgabe. Fertig. Und wenn die Datenhaltung nur in-memory ist, verringert sich der Umsetzungsaufwand dafür noch weiter. (Das ist der Fall, wenn die Entscheidung für Persistenz zunächst und temporär verneint wird.) Auf diese Weise kann der PO in kürzester Zeit Rückmeldung geben, ob ihm der allgemeine Fluss der Bedienung passt.

Der PO kann sogar bestimmen: All die anderen Entscheidungen, die zunächst negativ/minimal getroffen wurden, bleiben so. Anderes ist wichtiger, als nach einem ersten Erkenntnis-Inkrement noch eine Löschbestätigung zu implementieren oder auf die Tags aufzupassen und so fort. Er kann bestimmen, dass das, was nur ein Erkenntnis-Inkrement hatte sein sollen, doch ein Wert-Inkrement ist. Nicht 100 Prozent des ursprünglichen Wertes wurde erreicht, aber zum Beispiel 80 Prozent. Das ist gut genug für eine Auslieferung, die schon Nutzen stiftet. (Wenn zumindest die Entscheidung für Persistenz nun bejaht und der In-Memory-Provider durch einen persistenten ausgetauscht wird.)

„Gut genug“ denken zu können ist auch ein Ergebnis von Loslassen.

Das SRP kommt als Prinzip für die Strukturierung daher. Es ist eines der fünf Clean-Code-SOLID-Prinzipien. Doch woher kommen die Verantwortlichkeiten, die nach dem SRP sauber „verpackt“ werden sollen? Es sind Aspekte der Anforderungen. Deshalb beginnt der Einsatz des SRP schon während der Anforderungsanalyse. Es leitet Ihre Fragen. Es fordert Sie auf, tiefer und tiefer zu bohren und noch genauer hinzuschauen, ob nicht noch eine Entscheidung unentdeckt und/oder nicht explizit entschieden ist. Sobald Sie bemerken, eine Annahme zu treffen, halten Sie inne! Nehmen Sie nicht an, sondern legen Sie die Entscheidung dem PO vor.

Damit meine ich natürlich Entscheidungen rund um die Ausprägung von Verhalten der Software. Technische Detailentscheidungen – „Sollte hier LINQ benutzt werden?“ oder „Benutzen wir *class*, *struct* oder *record*?“ – sind nicht für den PO relevant. Auf sie bezieht sich das SRP zwar auch, doch die Entscheidung müssen Sie beziehungsweise Ihr Entwicklerteam treffen.

Die Features einer Software

Der Begriff „Feature“ ist Ihnen für Anforderungen geläufig. „Das Feature X der Anwendung ist gut gelungen“ oder „Das

Feature Y sollte nun endlich umgesetzt werden ...“ sind Aussagen, die Sie sicher auch schon getätigt haben. Doch was ist mit „Feature“ gemeint? In meiner Wahrnehmung geht es allermeistens um eine funktionale Anforderung. Und ich würde sogar sagen: Eine funktionale Anforderungen im Rahmen einer Interaktion.

Deshalb möchte ich den Begriff wie folgt formalisieren: Ein Feature ist ein Inkrement innerhalb des Inkrements, für das ein Entry Point (beziehungsweise eine seiner CQS-Funktionen) steht. Realisierte Features sind umgesetzte Entscheidungen im Rahmen des Scopes. Der Scope sind die Anforderungen an einen Entry Point.

In der Umsetzung bekommt jedes Feature einen Bezeichner, am besten durch Kapselung in einer Funktion. Bei größeren Features kann dafür aber auch eine Klasse oder ein noch höheres Modul stehen. Features sind also implementierte Verantwortlichkeiten des SRP.

Nochmal als Beispiel die Aufgabenverwaltung: Dass eine Aufgabe gelöscht werden kann, würde ich nicht als Feature bezeichnen. Das ist eine Anforderung, die durch eine Interaktion in einem Dialog beantwortet wird.

Innerhalb der Interaktion „Aufgabe löschen“ jedoch gibt es Features, zum Beispiel:

- Bestätigungsabfrage
- Verweigerung des Löschens, wenn es noch abhängige Aufgaben gibt
- Löschen eines Tags, das die Aufgabe als einzige enthält
- Soundeffekt abspielen

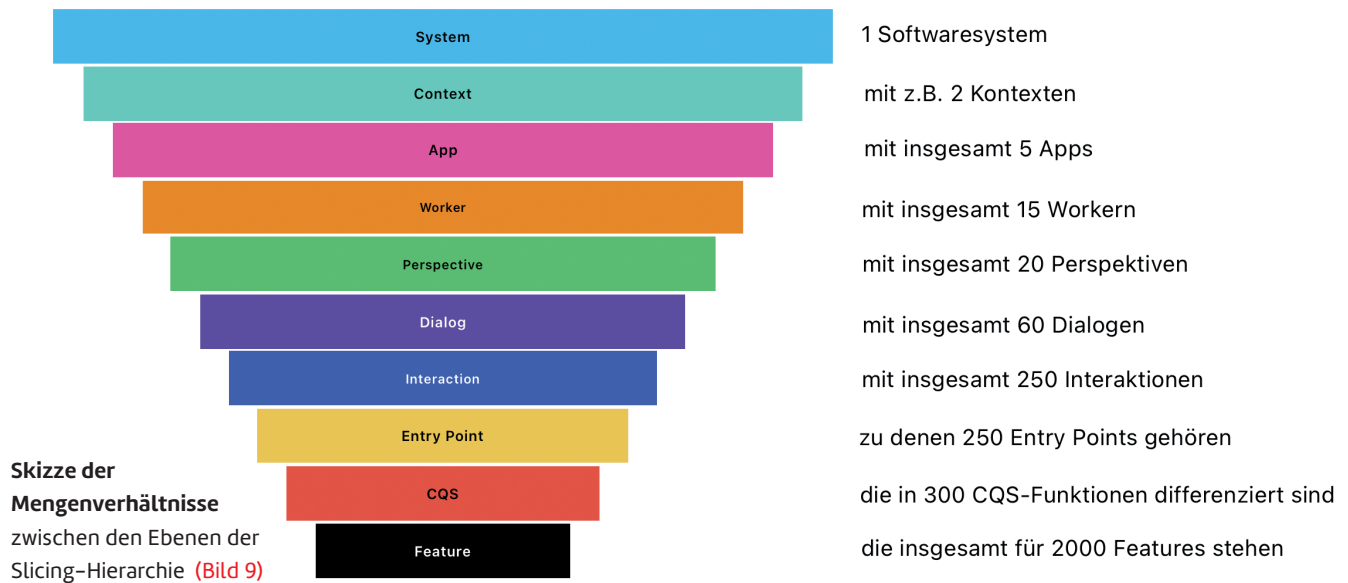
```
void btnDelete_Click(...) {
    const taskId = ...;
    if (ui.ConfirmDeletionRequest()) {
        if (processor.Delete(taskId, out explanation) == false) {
            ui.ReportFailure(explanation)
        }
    }
}

boolean Delete(string taskId, out string explanation = "") {
    if (provider.DependentTasksExist(taskId)) {
        explanation = "Cannot delete while dependent tasks exist!";
        return false;
    }

    const tags = provider.Load(taskId).tags;
    provider.DeleteTask(taskId);
    DeleteOrphanedTags(tags)
}
```

Alle Features sind gekapselt in Funktionen (Bild 8)

Bild 8 zeigt mit einer Code-Skizze, wie Features in Funktionen gekapselt sein könnten. Präfixe für Funktionsnamen deuten eine Modularisierung an. Auf diese Weise könnten sogar schon alle Features repräsentiert sein – ohne sie implementieren zu müssen. Außer der Funktion *provider.DeleteTask()* könnten alle nur minimale Rümpfe sein. Auf diese Weise würden Sie Anforderungen im Code dokumentieren, ohne schon Implementationsaufwand treiben zu müssen. (Aber natürlich gilt auch hier: Vorsicht vor Premature Optimization!) ►



Fazit

Meine klare Empfehlung zur Abrundung Ihrer Anforderungsanalyse mit Slicing: Entwickeln Sie einen Spürsinn für Features. Seien Sie nicht mit Entry Points als Endpunkten der Anforderungserlegung zufrieden. Schauen Sie unter die Haube der Entry Points beziehungsweise CQS-Funktionen. Dort finden Sie immer zahlreiche Features, die Sie mehr oder weniger unabhängig voneinander implementieren können.

Das SRP leitet Sie bei der Codierung, indem es Ihnen nahelegt, jedes Feature mit einem Bezeichner zu identifizieren – am besten ist das mindestens eine Funktion.

Und gegenüber dem PO können Sie den Aufwand bis zur Lieferung des nächsten Inkrements mittels Features fein steuern. Peilen Sie als Iterationsdauer maximal 16 Stunden an. In dieser Zeit konzentrieren Sie sich auf Inkremente gebündelt aus einem oder mehreren Features einer Interaktion.

Lassen Sie sich nicht wild machen durch Bettelei des PO nach Wert. Konzentrieren Sie sich auf den Mangel, den Sie spüren. Wenn für Wert noch zu wenige Informationen vorliegen, liefern Sie Inkremente für den Erkenntnisgewinn. Wert wird sich zur rechten Zeit schon einstellen. Keine Sorge.

Features machen den Zerlegungsbaum des Slicing natürlich sehr „blattlastig“. Ich versuche in **Bild 9** einmal eine ganz überschlägige Rechnung:

- Überschaubar ist noch die Menge der Worker innerhalb eines Softwaresystems. Bis dahin mag eine Breadth-first-Zerlegung möglich sein.
- Anschließend wird es schon sehr detailliert, und der Fokus sollte auf eine App in einem Kontext gehen.
- Innerhalb einer App lohnt sich wiederum die Priorisierung einer Perspektive mit wenigen Dialogen und ihren Interaktionen.
- Denn auf der untersten Ebene ist die Zahl der Features mindestens zwei Zehnerpotenzen größer als die Zahl der Dialoge. (Wahrscheinlich ist diese Abschätzung sogar noch untertrieben: Die Zahl der Features „geht durch die Decke“, wenn Sie wirklich genau hinhören und nicht nur größere,

sondern auch kleine, schwer erkennbare Entscheidungen aufdecken. So viel wird impliziert, vorausgesetzt ...)

Also: Augen und Ohren auf bei der Anforderungsanalyse! Lassen Sie sich nicht mit User Stories und Use Cases „abspeisen“. Fordern Sie viel konkretere Entscheidungen des PO, die Ihnen zunächst präzise spezifizierte Entry Points und dann einen Fächer an Features je Entry Point liefern, mit dem Sie das inkrementelle Vorgehen für schnelle Rückmeldungen fein justieren können. Das ist pragmatische Agilität. ■

[1] Ralf Westphal, *Strukturiert zerlegen*, dotnetpro 4/2024, Seite 42 ff., www.dotnetpro.de/A2404Slicing

[2] Ralf Westphal, *Den Stier bei den Hörnern packen*, dotnetpro 5/2024, Seite 26 ff., www.dotnetpro.de/A2405Slicing

[3] Ralf Westphal, *Das Messer richtig ansetzen*, dotnetpro 6/2024, Seite 34 ff., www.dotnetpro.de/A2406Slicing

[4] Ralf Westphal, *Die Benutzerschnittstelle treibt das Slicing*, dotnetpro 7/2024, Seite 44 ff., www.dotnetpro.de/A2407Slicing

[5] Ralf Westphal, *Grobe Schnitte durch die Anforderungen*, dotnetpro 8/2024, Seite 30 ff., www.dotnetpro.de/A2408Slicing

[6] Ralf Westphal, *The Single Responsibility Principle*, www.dotnetpro.de/SL2409Slicing1



Ralf Westphal

ist Trainer, Berater und Mitgründer der Clean Code Developer Initiative (<https://clean-code-developer.de>). Seine Schwerpunkte sind dauerhaft hohe Produktivität für die Softwareentwicklung und zukunftsfähige Teamorganisation.

<https://ralfw.de>

dnpCode

A2409Slicing